

Figure 1: A fixed size window

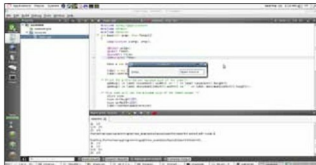


Figure 2: Horizontal layout

```
hbox = new QHBoxLayout(&widget);
label = new QLabel(&widget);
label->setText("Linux");

/* Print the preferred and minimum size of the label*/
QDebug() << label->sizeHint().width() << " " << label-
>sizeHint().height();
QDebug() << label->minimumSizeHint().width() << " " << label-
>minimumSizeHint().height();

/* This code will set the minimum size of the label widget */
QSize size;
size.setHeight(20);
size.setWidth(200);
label->setMinimumSize(size);

/* Add QLineEdit Widget */
line = new QLineEdit(&widget);

/* Print the preferred and minimum size of the line*/
QDebug() << line->sizeHint().width() << " " << line-
>sizeHint().height();
QDebug() << line->minimumSizeHint().width() << " " <<
line->minimumSizeHint().height();

/* Add widgets to the HBOX */
hbox->addWidget(label);
hbox->addWidget(line);
widget.show();
```

```
return a.exec();
}
```

Let us examine what the above code actually does. We first created a *QWidget* object named *widget*. Then a *QHBoxLayout* object *hbox* was created with the *widget* as its parent. Further, we created more child widgets of the parent *widget*, not the *hbox*, as the *BoxLayout* cannot be the parent of any widget. And, finally, the child widgets were added to the *hbox*.

```
hbox->addWidget(label);
hbox->addWidget(line);
```

What was done for the horizontal layout can be done for the *QVBoxLayout* too.

In the above sample of code, you will notice that you need not worry about the co-ordinates of the child widgets. They are arranged automatically. But the important thing is the size management of the widgets inside the *BoxLayout*. When you create a widget, it comes with a default size, which is the minimum size. When you add widgets to a *BoxLayout*, the issue of the space sharing may crop up. A widget cannot be reduced beyond its minimum size. You can retrieve the default and the minimum size of the widget as shown:

```
QDebug() << label->sizeHint().width() << " " << label->sizeHint().
height();
QDebug() << label->minimumSizeHint().width() << " " << label-
>minimumSizeHint().height();
```

In the above two lines, the default and the minimum size of the label have been printed. The functions for this are *sizeHint* and *minimumSizeHint*. Instead of *sizeHint*, *sizeHint.width()* and *sizeHint.height()* can be used to check the width and the height of the widgets. If required, you can change this size as follows.

```
QSize size;
size.setHeight(20);
size.setWidth(200);
label->setMinimumSize(size);
```

I think the above code is easy to understand and needs no more explanation.

Stretch

After the compilation of *BoxLayout*, let us look at how to stretch the layout:

```
#include <QtGui/QApplication>
#include <QtGui>
#include <QtCore>
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    QWidget w;
    QLineEdit *line;
```